

Step 2 — Writer ↔ Reviewer Loop — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

`agents/writer_agent.py`

```
from agents.writer_chain import build_writer_chain

_chain = build_writer_chain()

def run_writer(notes, feedback=None):
    if feedback:
        prompt_notes = notes + "\n\nPrevious feedback to address:\n" + feedback
    else:
        prompt_notes = notes
    return _chain.invoke({"notes": prompt_notes})
```

`agents/reviewer_agent.py`

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class ReviewVerdict(BaseModel):
    approved: bool
    feedback: str

def run_reviewer(draft):
    model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(ReviewVerdict)
    return model.invoke("Review this draft for clarity and accuracy:\n" + draft)
```

`main.py`

```
from agents.writer_agent import run_writer
from agents.reviewer_agent import run_reviewer
```

```
MAX_ROUNDS = 3
notes = "Electric bikes cost $800-3000. Battery range 20-60 miles."
feedback = None

for i in range(MAX_ROUNDS):
    draft = run_writer(notes, feedback)
    verdict = run_reviewer(draft)
    print(f"Round {i+1}, approved={verdict.approved}")
    if verdict.approved:
        break
    feedback = verdict.feedback

print(draft)
```

This works. It reuses Step 1's chain via a module-level `_chain`, which is fine, though building the chain fresh inside `run_writer` on every call (Intermediate version) is a little cleaner for testing. Nothing here notices if the Reviewer gives the same feedback twice in a row — it just spends every remaining round anyway.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a real, testable loop function

`agents/writer_agent.py`

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

WRITER_PROMPT = (
    "You are a content writer. Write a short, clear draft (3-5 paragraphs) "
    "based only on these research notes. Do not invent facts not in the notes.\n\
    "Research notes:\n{notes}\n\n"
    "{feedback_section}"
)

def run_writer(notes: str, feedback: str | None = None) -> str:
    feedback_section = ""
    if feedback:
        feedback_section = f"Revise to address this reviewer feedback:\n{feedback}"

    prompt = ChatPromptTemplate.from_template(WRITER_PROMPT)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
    chain = prompt | model | StrOutputParser()
    return chain.invoke({"notes": notes, "feedback_section": feedback_section})
```

`agents/reviewer_agent.py`

```
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI

class ReviewVerdict(BaseModel):
    approved: bool = Field(description="True only if clear, accurate to the notes")
    feedback: str = Field(description="Specific, actionable feedback if not approved")

def run_reviewer(draft: str) -> ReviewVerdict:
```

```

model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output
return model.invoke(
    "Review this draft against: clarity, factual grounding in the notes, comp
    f"Draft:\n{draft}"
)

```

main.py

```

from agents.writer_agent import run_writer
from agents.reviewer_agent import run_reviewer, ReviewVerdict

MAX_ROUNDS = 3

def run_writer_reviewer_loop(notes: str) -> tuple[str, ReviewVerdict]:
    feedback: str | None = None
    verdict: ReviewVerdict | None = None
    draft = ""

    for round_num in range(MAX_ROUNDS):
        draft = run_writer(notes, feedback)
        verdict = run_reviewer(draft)
        if verdict.approved:
            return draft, verdict
        feedback = verdict.feedback

    return draft, verdict # ran out of rounds; return last attempt, unapproved

if __name__ == "__main__":
    notes = "Electric bikes cost $800-3000. Battery range 20-60 miles."
    final_draft, final_verdict = run_writer_reviewer_loop(notes)
    print(final_draft)
    print("Approved:", final_verdict.approved)

```

Difference from Basic: the loop logic is pulled into its own testable function, `run_writer_reviewer_loop`, instead of living directly in a script's top level — this is exactly what `test_writer_reviewer.py` needs to import and call. Full type hints, including the `tuple[str, ReviewVerdict]` return type, which tells any caller (including Step 5's Supervisor, later) exactly what it gets back. `temperature=0` on the Reviewer keeps its judgments consistent between runs. Still missing: any way to tell "gave up because it wasn't improving" apart from "gave up because rounds ran out" — both return the same shape, `(draft, unapproved_verdict)`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — no-progress detection + round history

```
from dataclasses import dataclass, field
from agents.writer_agent import run_writer
from agents.reviewer_agent import run_reviewer, ReviewVerdict

MAX_ROUNDS = 3

@dataclass
class LoopResult:
    draft: str
    verdict: ReviewVerdict
    status: str # "approved" | "no_progress" | "max_rounds"
    history: list[tuple[str, ReviewVerdict]] = field(default_factory=list)

def run_writer_reviewer_loop(notes: str, max_rounds: int = MAX_ROUNDS) -> LoopResult:
    feedback: str | None = None
    previous_feedback: str | None = None
    history: list[tuple[str, ReviewVerdict]] = []
    draft = ""
    verdict: ReviewVerdict | None = None

    for round_num in range(max_rounds):
        draft = run_writer(notes, feedback)
        verdict = run_reviewer(draft)
        history.append((draft, verdict))

        if verdict.approved:
            return LoopResult(draft, verdict, "approved", history)

        if previous_feedback is not None and verdict.feedback == previous_feedback:
            return LoopResult(draft, verdict, "no_progress", history)

        previous_feedback = verdict.feedback
        feedback = verdict.feedback

    return LoopResult(draft, verdict, "max_rounds", history)
```

```

if __name__ == "__main__":
    result = run_writer_reviewer_loop(
        "Electric bikes cost $800-3000. Battery range 20-60 miles."
    )
    print(result.draft)
    print("Status:", result.status, "| Rounds run:", len(result.history))

```

Expected output (a normal case): a draft, then `Status: approved | Rounds run: 1 or 2`. On a deliberately-impossible-to-satisfy case where the Reviewer repeats the same complaint, you'd see `Status: no_progress | Rounds run: 2` — stopped one round earlier than `max_rounds` would have allowed, because continuing clearly wasn't helping.

Approach 2 — same idea, exposed as a plain function return instead of a dataclass

```

def run_writer_reviewer_loop(notes: str, max_rounds: int = MAX_ROUNDS):
    feedback = None
    previous_feedback = None
    history = []
    draft = ""
    verdict = None

    for round_num in range(max_rounds):
        draft = run_writer(notes, feedback)
        verdict = run_reviewer(draft)
        history.append((draft, verdict))

        if verdict.approved:
            return draft, verdict, "approved", history
        if previous_feedback is not None and verdict.feedback == previous_feedback:
            return draft, verdict, "no_progress", history

        previous_feedback = verdict.feedback
        feedback = verdict.feedback

    return draft, verdict, "max_rounds", history

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's loop only ever tells you `verdict.approved` — `True`, or `False` with no explanation of why it gave up. Both Advanced approaches add the same two things on top: a `history` list (every round's draft and verdict, useful for debugging and for a later observability step), and a third status, `"no_progress"`, that stops the loop early when the Reviewer's feedback repeats verbatim instead of burning every remaining round on a redraft that already failed to help once. Approach 1 packages the result as a small `LoopResult` dataclass, which is easier for a caller to read (`result.status`,

`result.history`) and easier to extend later without breaking every call site's unpacking. Approach 2 is the plain-tuple version — no new class to define, but every caller has to remember the order of 4 return values, and adding a 5th one later means changing every call site.

Which one should you actually write? Approach 1's dataclass. Step 5's Supervisor is going to call this exact loop from inside a bigger system, and by then you'll likely also want to log `result.status` and `result.history` somewhere — a named dataclass with clear fields survives that kind of growth much better than a 4-item tuple. The no-progress check itself (in either approach) is worth keeping regardless: it's a small, cheap addition that turns "wasted every remaining round" into "recognized it was stuck and said so," which is exactly the difference between a demo loop and one you'd trust in a system a real caller depends on.